Midterm Project: Report

Who Wrote It? Identifying LLMs from Their Responses

AlMahanad AlMahdhour (individual project)

1. Introduction

In this project, my aim is to find out whether the model can be able to distinguish the author of the text written by the LLM. I have created my dataset consisting of (LLM_name, LLM_input, LLM_output) where I have used three LLMs: Claude, GPT, and DeepSeek. I have trained the two required models, i.e., CNN and RNN (bidirectional LSTM) to predict the LLM of each response. I have answered RQ1 and RQ2 and also done RQ3 and RQ4 as extra credit.

2. Dataset

Prompts: For the creation of prompts, I used Claude Opus 5.5 (medium effort) and created 600 prompts for 3 different categories, 200 for each category. The coding category has 40 tasks in 5 different languages (Python, JavaScript, Java, C++ and Go), math has 180 problems with random numbers and 20 proof and concept problems and writing has 20 different types such as poems, emails and short stories for 10 topics. All prompts were written for this project, so there is no private or copyrighted data.

Collection: Each of the three LLMs was fed the same set of 600 prompts. In each case, 10 subagents in each model harness were spawned to answer 60

prompts each, so that the answer is not truncated by trying to respond to too many prompts at once.

Claude: I used Claude Opus 5.5 through Claude Code.

GPT: I used Codex with GPT 6 Luna on the free version :), and saved the answers to a CSV file. My first run had so many short answers and so I reran it with a different prompt and it was longer, but still shorter than other models

DeepSeek: I used DeepSeek 4.1 Flash through their harness and API, it cost a dollar ;) and it also saved it into a CSV file.

Cleaning: I checked each file for any missing or duplicate answers, any blank answers, coded answers without code, and truncated code, and all the final files have cleared all of these checks. The dataset.csv of the final data contains 1,800 records (600 for each LLM), which means we got ourselves a balanced sheet we can test on.

3. Methods

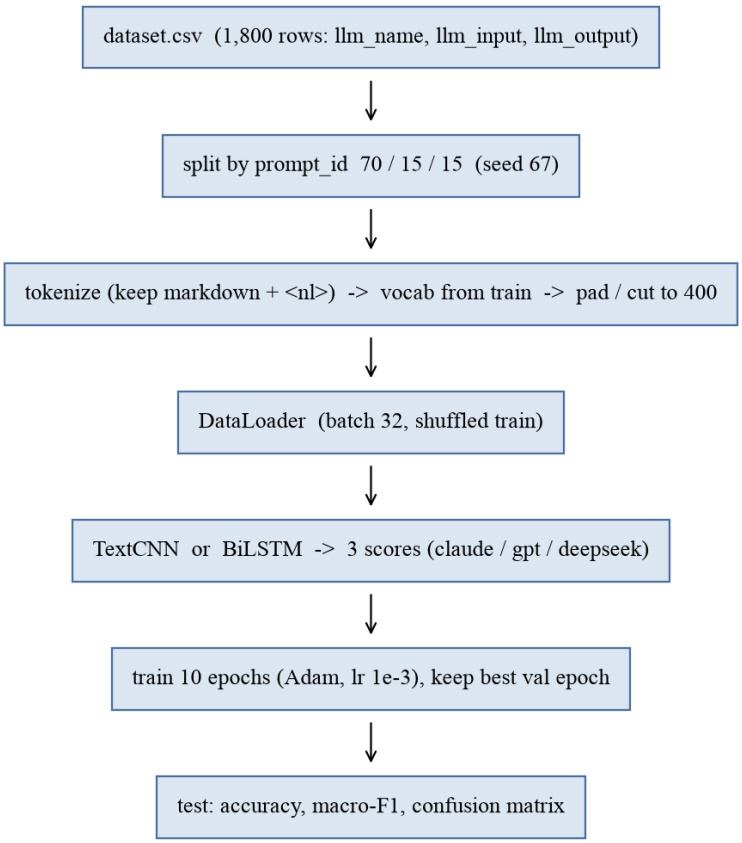


Figure 1: Pipeline used for every experiment.

Model Details:

Preprocessing: I used a 70/15/15 split for training, validation, and testing (1,260 / 270 / 270 rows, seed 67). Instead of splitting by row, I split by prompt, thus keeping all three responses to the same prompt together, and making sure that the model does not see any test prompt during training (Master Claude told me this is better). Tokenization involved lowering case for all texts, keeping words, punctuation, and markdown characters such as # and *, and using a <nl> token for line breaks (original text used \n), since

formatting proved to be an important cue. Vocabulary was constructed based on the training text alone, and all responses were truncated/padded to 400 tokens, which is enough to contain the entire response 64% of the time, and I tested both for accuracy and the difference is like 1-2% so not worth to double the computational usage for 2% improvement.

CNN: The CNN model I used was recommended by Claude and was inspired by Kim (2014). Firstly, each token is converted to a vector with 128 numbers (embedding). Secondly, three convolution layers process the text in windows of 3, 4, and 5 tokens. These have 100 filters that will learn to recognize small patterns such as bold headings or phrases. Max pooling keeps the best match for each filter regardless of its location within the text. Finally, after dropout (0.5), a linear layer produces a score for each of the 3 LLMs, and the highest score is chosen as the output.

LSTM: I implemented the model using the LSTM architecture that Claude also suggested :). The first step is to perform the same embedding of the tokens as in the CNN, which consists of 128 numbers. Afterward, a bidirectional LSTM takes the text token by token, first from the beginning to the end and then from the end to the beginning, while remembering 128 numbers in each direction. I used packed sequences because this way, the LSTM skips the padding and processes only the actual text. The last step is concatenation of the memories from both directions and creation of the summary consisting of 256 numbers. After the dropout (0.5), the linear layer assigns the score to each of the 3 LLMs. In total, the model has 1,752,579 parameters.

Training Details:

I used Adam as the optimizer with a learning rate of 0.001 and cross-entropy loss. I ran the training for 10 epochs using batch sizes of 32, and shuffled the training dataset every epoch to ensure that each batch contains all three LLMs. I used gradient clipping with a threshold of 1.0 to ensure the stability of the LSTM model. After every epoch, I calculated the validation accuracy, saved the best epoch, and evaluated that model once using the test dataset. I collected reporting on accuracy, macro-F1, and the confusion matrix. I have set the random seed, but on my Mac GPU, there is a variance of around 1 point between the runs because I had few variances and Claude said MPS is the reason.

4. Results

RQ1: Can we identify which LLM generated a response?

Yes. The CNN had 88.1% test accuracy (macro-F1 0.881) based on the responses alone, while the LSTM scored 70.7% (macro-F1 0.709), versus 33.3% by random chance.

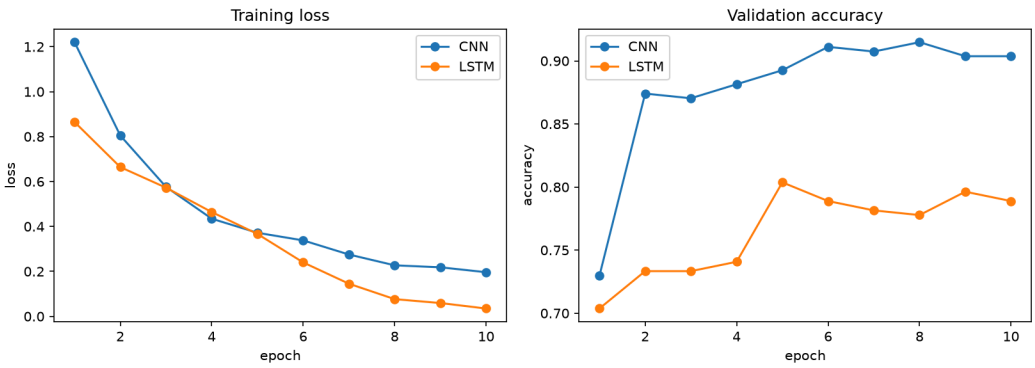


Figure 2: Training loss and validation accuracy per epoch (RQ1).

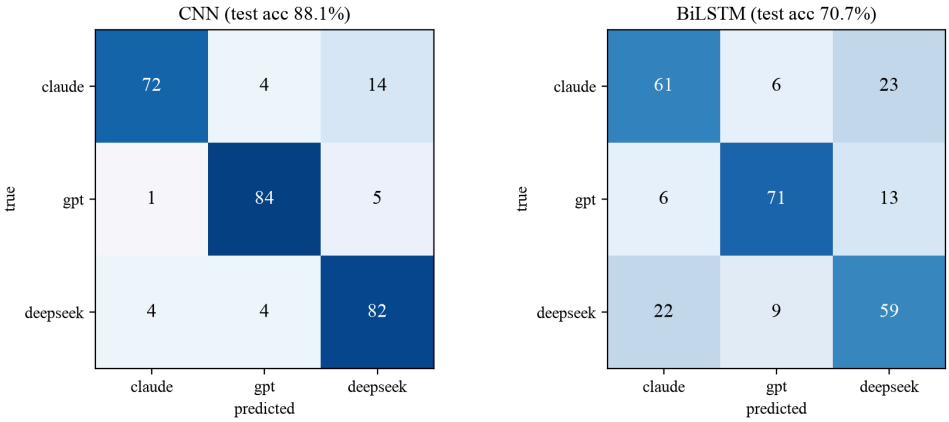


Figure 3: Test confusion matrices (RQ1).

GPT was the easiest to detect (84 out of 90 correct using the CNN). Errors were mainly due to the confusion between Claude and DeepSeek, as both produce long responses full of markdowns and also I think DeepSeek was trained partially on Claude :). The LSTM performed poorly because it was overfitting; its training loss dropped to 0.035, but its validation accuracy plateaued at epoch 5 (Figure 2). I believe that the CNN is better suited for this task because it only needs to detect short indicative patterns in the text, whereas the LSTM needs to remember things throughout 400 tokens.

RQ2: Does the user's prompt help identify the LLM?

Input	CNN	BiLSTM
Output only	88.1%	70.7%
Input only	33.3%	33.3%
Input + output	87.8%	63.0%

i) Predicting from the prompt alone: No. In both cases, the accuracy was 33.3%, which amounts to pure guessing. It is logical that all LLMs received the same prompts, meaning each prompt occurs only once for each label and there is nothing to learn. The optimal loss value is $\ln 3 \approx 1.099$, and the LSTM's loss stagnated precisely at this value – 1.104.

ii) Does the input help? Not really. The CNN performance remained almost unchanged (87.8% compared to 88.1%) and even the LSTM performance declined (63.0%) because the prompt at the beginning of the text does not contain any useful information for the LSTM.

iii) Why input-only could work elsewhere: In real-world datasets, users ask various LLMs different types of questions (for instance, coders use one model while students another), so the prompt itself can reveal the answer. So if I had used certain prompts like DeepSeek and the others on GPT the model would probably memorize the type of questions that GPT gets asked and DeepSeek gets asked too.

5. In-depth Analyses

RQ3: Do LLM fingerprints generalize across tasks?

For each domain, I trained on the other two domains and tested on all 600 questions from the unseen domain (for instance, trained on coding and math domains and tested on writing, as seen in the project description). This was compared with the models trained on all three domains ("seen").

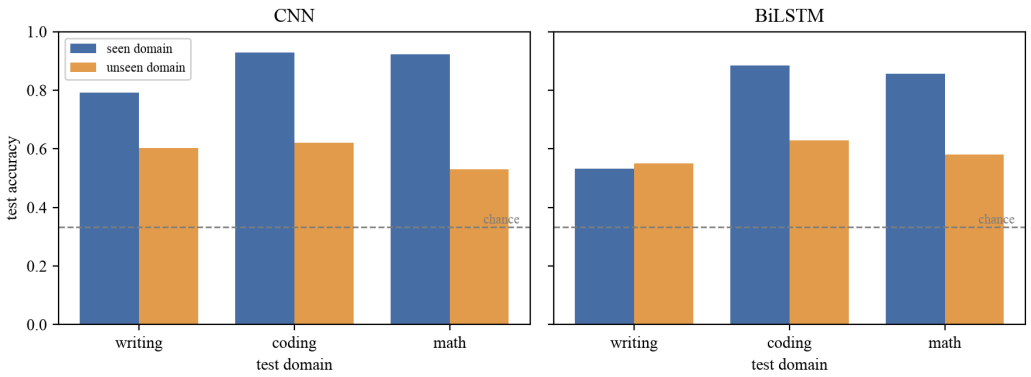


Figure 4: Test accuracy on a seen vs unseen domain (RQ3).

i) Does performance drop? Yesss, by a large margin. The CNN scored 19 to 39 points lower (e.g., math dropped from 92.2% to 53.0%), while the LSTM scored 28 points lower. However, all scores were higher than random guesses (53% to 63%).

ii) Most consistent LLM: GPT was the most consistent, maintaining its LSTM F1 between 0.63 and 0.73 in all three out-of-domain sets, while Claude was the least consistent (F1 0.43 to 0.70), and continued being mistaken for DeepSeek :).

iii) General or task-specific? Task-specific for the most part, but with some general fingerprint too. The large drops indicate that the models learned mostly how each LLM produces code, mathematics, or emails in particular. The reason it is still better than guessing is because there is some general fingerprint in addition, such as answer length and formatting preferences. Furthermore, the validation accuracy was consistently high (from 74% to 92%) in all experiments, indicating no warning about bad performance in a new domain.

RQ4: What characteristics distinguish different LLMs?

Average per answer	Claude	DeepSeek	GPT
Words	222.8	200.1	96.7
Lines	43.9	33.4	14.1
Headers / bullets	2.2 / 2.2	1.2 / 1.8	0.2 / 0.3
Bold phrases	5.0	3.7	0.8
LaTeX symbols	5.2	7.3	0.5
Sentence length (words)	25.7	24.9	19.5
Type-token ratio (first 100 words)	0.63	0.62	0.68
Word length (letters)	4.36	4.33	4.38

Claude uses the most structure (most lines, headings, bullets, and bolding), DeepSeek uses the most math symbols, and GPT gives short and direct answers (half as many words, minimal markdown, shorter sentences, and more varied vocabulary). The average word length is pretty much identical for all three.

i) Signals the classifier uses: I did train a random forest on only these 13 counts as suggested by Claude and it got 80.0%. The most important variables turned out to be word count, line count, bold and sentence length.

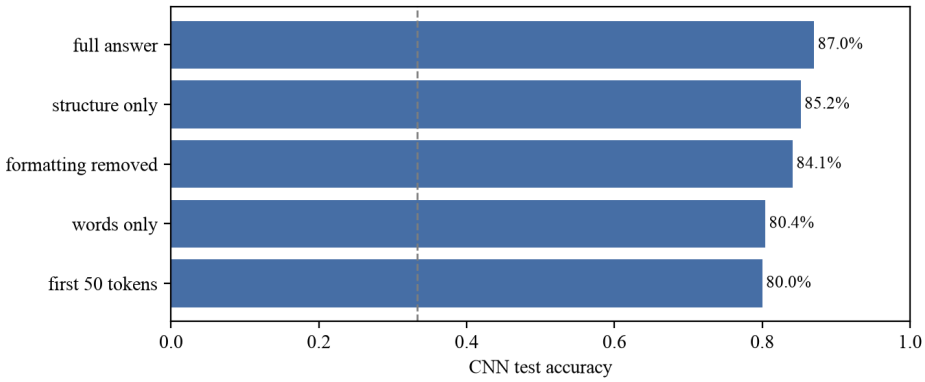


Figure 5: CNN accuracy after removing signals (RQ4).

ii) Linguistic, structural or both? Both. I trained the CNN again using all "w"s in place of the words to remove the content and retain just the structure, and it scored 85.2%. Just using the words without any punctuation or formatting gave me 80.4%.

iii) Does removing signals hurt? Not much. Stripping the formatting only reduced accuracy by 2.9 points, while restricting to the first 50 tokens (so that length no longer helps) reduced accuracy by 7.0 points. I suspect that this is due to the fact that the LLMs differ in multiple ways simultaneously, so stripping out one clue leaves other clues intact.

6. Lessons & Experience

How you get the data is as important as the model. I got each LLM differently, and GPT's responses were shorter, which means that some of the

differences could be caused by the way I got the data and not GPT itself. In the future, I will give all models the same instructions, one prompt per chat.

The fact that I gave all LLMs the same prompts made the results much easier to interpret, and the division into prompts helped to keep the test set balanced.

Simple models work very well indeed. CNN outperformed LSTM by 17 points and trained around 8 times faster, while even 13 simple counts scored 80%. A high validation score does not guarantee success on other tasks (RQ3). In addition, with only 270 test responses, a difference of 1-3 points might just be random chance, so running a few seeds would increase reliability.

Collaborations: None. I used Claude to help with brainstorming, the model architectures, debugging, writing the prompts and collecting Claude's answers, which the project guidelines allow.

7. References

Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1746–1751).